



What Is an Algorithm and How Does Big O Notation Work?

Every computer program that runs on your device has a specific set of instructions, which are executed in a specific order to complete a task.

The task could be sorting a set of numbers, modifying an image, tracking inventory, or even running your favorite video game.

This is where algorithms come into play. An **algorithm** is a set of unambiguous instructions for solving a problem or carrying out a task.

You can think of algorithms as "recipes". When you cook, recipes list all the ingredients that you'll need, and provide step by step instructions on how to prepare a dish.

Equivalently, you can think of algorithms as "recipes" that tell computers exactly what should be done and how to do it.

Algorithms have two key characteristics:

- They cannot continue indefinitely. They must finish in a finite number of steps.
- Each step must be precise and unambiguous.

They may have zero, one, or more inputs, and generate one or more outputs.

The steps of an algorithm are independent from any programming language.

But to actually make them run on a computer, you need to implement them in a programming language, like Python or JavaScript.

If an algorithm is correct, the output for any valid input should match the expected output.

In addition to being correct, algorithms should also be efficient.

Algorithm efficiency can be measured in terms of how long they take to run and how much space they require in memory to complete the task.

Knowing an algorithm's efficiency is very important because it gives you an idea of how well it will perform as the input size grows.

For example, sorting 15 integers is not the same as sorting 1 million integers.

As the process grows in size and complexity, if the algorithm is not efficient enough to handle it, you might end up with a very slow computer program that may even crash the entire system.

That's why it's very important to develop and choose the most efficient algorithms possible.

This is where Big O notation becomes very important.

Big O notation describes the worst-case performance, or growth rate, of an algorithm as the input size increases.

The growth rate of an algorithm refers to how the resources it requires increase as the input size grows.

Big O notation focuses on the worst-case performance because this case is very important to understand how efficient the algorithm can be, even in the worst case scenario, regardless of the input.

Going back to our sorting example, sorting 1 million integers should intuitively take more time and resources than sorting 15 integers.

But how much more?



input size grows, based on the number of operations performed by the algorithm.

In Big O notation, we usually denote input size with the letter n . For example, if the input is a list, n would denote the number of elements in that list.

Constant factors and lower-order terms are not taken into account to find the time complexity of an algorithm based on the number of operations. That's because as the size of n grows, the impact of these smaller terms in the total number of operations performed will become smaller and smaller.

The term that will dominate the overall behavior of the algorithm will be the highest order term with n , the input size.

For example, if an algorithm performs $7n + 20$ operations to be completed, the impact of the constant 20 on the final result will be smaller and smaller as n grows. The term $7n$ will tend to dominate and this will define the overall behavior and efficiency of the algorithm.

Another example would be an algorithm that takes $20n^2 + 15n + 7$ operations to be completed. The term $20n^2$ will tend to dominate as n grows, so this algorithm would have a quadratic time complexity because the dominant term has n^2 .

Quadratic time complexity is one of many different types of time complexities that you can find in the world of algorithms.

Let's learn about some of the most common ones.

$O(1)$ is known as "Constant Time Complexity". When an algorithm has constant time complexity, it takes the same amount of time to run, regardless of input size.

For example, checking if a number is even or odd will always take the same amount of time, regardless of the number itself.

```
def check_even_or_odd(number):  
    if number % 2 == 0:  
        return 'Even'  
    else:  
        return 'Odd'
```

$O(\log n)$ is known as "Logarithmic Time Complexity". This means that the time required by the algorithm increases slowly as the input size grows. This is common in problems in which the size of the problem is repeatedly reduced by a constant fraction.

For example, a popular search algorithm called Binary Search has $O(\log n)$ worst-case time complexity. This is because it eliminates half of the remaining elements in each comparison, which makes it more efficient overall.

$O(n)$ is known as "Linear Time Complexity". The running time of algorithms with this time complexity increases proportionally to the input size.

For example, a `for` loop that iterates over all the elements of a list will perform more iterations as the number of list elements increases. If the list is doubled in size, the number of operations will approximately double as well.

```
for grade in grades: # grades is a list.  
    print(grade)
```

$O(n \log n)$ is known as "Log-Linear Time Complexity". This is a common time complexity of efficient sorting algorithms, like Merge Sort and Quick Sort.



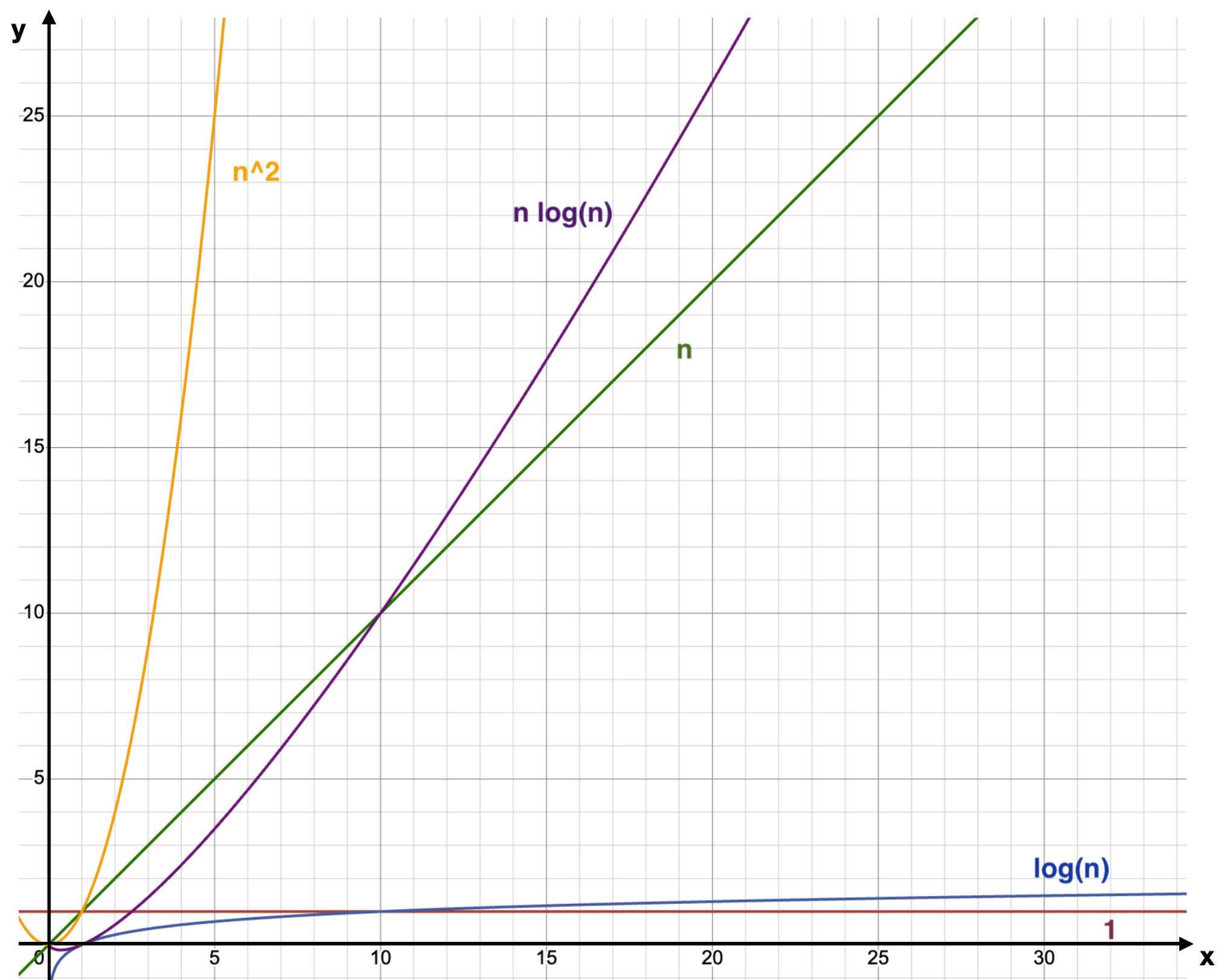
Nested loops are a common example of quadratic time complexity. The inner loop will perform n iterations for each one of the n iterations of the outer loop, resulting in n^2 squared iterations.

```
for i in range(n):  
    for j in range(n):  
        print("Hello, World!")
```

Other time complexities include "Exponential Time Complexity", denoted as $O(2^n)$, and "Factorial Time Complexity", denoted as $O(n!)$. Both are inefficient for real-world scenarios.

In this graph, you can compare the growth of the mathematical functions that represent the most common time complexities. Think of the x-axis (horizontal) as the input size and the y-axis (vertical) as the running time of the algorithm.

You can see that the Quadratic Time Complexity ($O(n^2)$) (yellow) grows much faster than the other ones, while the Constant Time Complexity ($O(1)$) (red) stays constant, even if the input gets larger.



Great. So far, you've learned about Big O notation in terms of time requirements, but this notation can also be applied to the context of space requirements.

In this context, it describes how the memory space required by the algorithm grows as the input size grows.



In contrast, the space required by algorithms with "Linear Space Complexity" $O(n)$ increases proportionally as the input size grows.

An example of this would be an algorithm that creates and stores a copy of a list of length n .

And finally, the space requirements of an algorithm with "Quadratic Space Complexity" $O(n^2)$ increase quadratically as the input size grows.

An example of this would be creating a 2D matrix, where the dimensions are determined by the input size, storing all possible pairs.

Algorithms are the building-blocks of computer programs, while Big O notation is a powerful framework for analyzing how efficient they are, based on how their time and space requirements in the worst-case scenario scale as the input size grows. Understanding their efficiency is very important for developing software that works efficiently in real-world scenarios.

Questions

Which of the following best describes an algorithm?

A specific programming language used to write code.

A set of step-by-step instructions designed to solve a problem or perform a task.

Correct!

A type of computer hardware component.

A software application used for developing and playing games.

What is the primary purpose of Big O notation in the context of algorithms?

To measure the exact time an algorithm takes to run on a specific computer in seconds.

To count the total number of lines of code in an algorithm.

To describe how the resource usage of an algorithm grows as the input size increases.

Correct!

To determine the best-case performance of an algorithm.

If an algorithm has a time complexity of $O(n)$, what does this mean about its performance?



Python Certification

Working with Common Data Structures

Correct!

The algorithm's running time remains constant regardless of the input size.

The algorithm's running time grows exponentially with the input size.

The algorithm's running time decreases as the input size gets larger.

Check your answer

Ask for Help